WriterINet: Writer Identification Using DenseNet201 and ResNet50 Feature Fusion

1. Introduction

Writer identification is a handwriting analysis task used to identify the author of handwritten text based on writing style patterns. In this project, a deep learning-based writer identification system named WriterINet was implemented using transfer learning and feature fusion techniques.

The system uses DenseNet201 and ResNet50 pretrained convolutional neural networks to extract handwriting features from the IAM Handwriting Dataset. These extracted features are fused together and classified using an Artificial Neural Network (ANN).

The project was implemented completely using PyTorch on Kaggle GPU.

2. Objective

The main objectives of this project are:

- To build a writer identification system using deep learning
- To use transfer learning for handwriting feature extraction
- To combine DenseNet201 and ResNet50 features
- To classify handwritten samples into writer classes
- To evaluate the system using confusion matrix and classification metrics

3. Dataset Used

The IAM Handwriting Dataset was used in this project.

Dataset Information

Parameter	Value
Dataset Name	IAM Handwriting Dataset
Total Samples	115,320

Total Writers	657
Data Type	Handwritten Word Images
Image Format	PNG
Annotation Format	XML

The dataset contains handwritten word images along with XML annotation files containing writer IDs.

4. Dataset Processing

The dataset was first extracted using Python tarfile extraction.

Two files were extracted:

- words.tgz
- xml.tgz

The XML files were parsed using xml.etree.ElementTree to extract:

- writer ID
- word image path

A dataframe was created containing:

- image_path
- writer_id

The writer IDs were converted into numerical labels using LabelEncoder.

5. Train-Test Split

The dataset was divided into:

- 80% training data
- 20% testing data

Stratified splitting was used to maintain equal writer distribution across train and test sets.

6. Image Preprocessing

The following preprocessing operations were applied:

- Resize images to 128 × 256
- Convert images to grayscale with 3 channels
- Convert images into tensors

PyTorch transforms were used for preprocessing.

7. Deep Learning Models Used

Two pretrained CNN models were used for feature extraction.

7.1 DenseNet201

DenseNet201 was used as a feature extractor by removing its final classification layer.

The extracted DenseNet feature size was:

1920 dimensions

7.2 ResNet50

ResNet50 was also used as a feature extractor by removing the final fully connected layer.

The extracted ResNet feature size was:

2048 dimensions

8. Feature Fusion

The features extracted from DenseNet201 and ResNet50 were concatenated together.

Final fused feature size:

3968 dimensions

This feature fusion approach combines complementary handwriting representations from both CNN models.

9. ANN Classifier

A fully connected ANN classifier was implemented for writer classification.

ANN Architecture

Layer	Configuration
Input Layer	3968 neurons
Hidden Layer 1	1024 neurons
Hidden Layer 2	512 neurons
Output Layer	657 classes

Activation Function

- ReLU

Regularization

- Dropout

Loss Function

- CrossEntropyLoss

Optimizer

- Adam Optimizer

10. Feature Caching Optimization

Initially, end-to-end training was attempted where CNN features were extracted during every epoch.

This process was computationally expensive.

To optimize training:

1. CNN features were extracted once
2. Features were saved as NumPy arrays
3. ANN was trained separately using cached features

This significantly reduced training time.

Saved feature files:

- X_train.npy
- y_train.npy
- X_test.npy
- y_test.npy

11. Training Details

Parameter	Value
Epochs	30
Batch Size	256
Learning Rate	0.001
Optimizer	Adam
Device	Kaggle GPU

The model was trained for 30 epochs using cached features.

12. Experimental Results

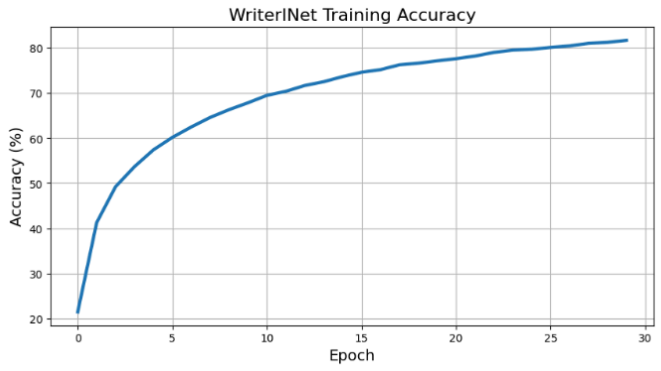
Final Results

Metric	Value
Training Accuracy	~81%
Test Accuracy	61.66%

13. Training Accuracy Curve

The training accuracy increased continuously during training, reaching approximately 81% after 30 epochs.

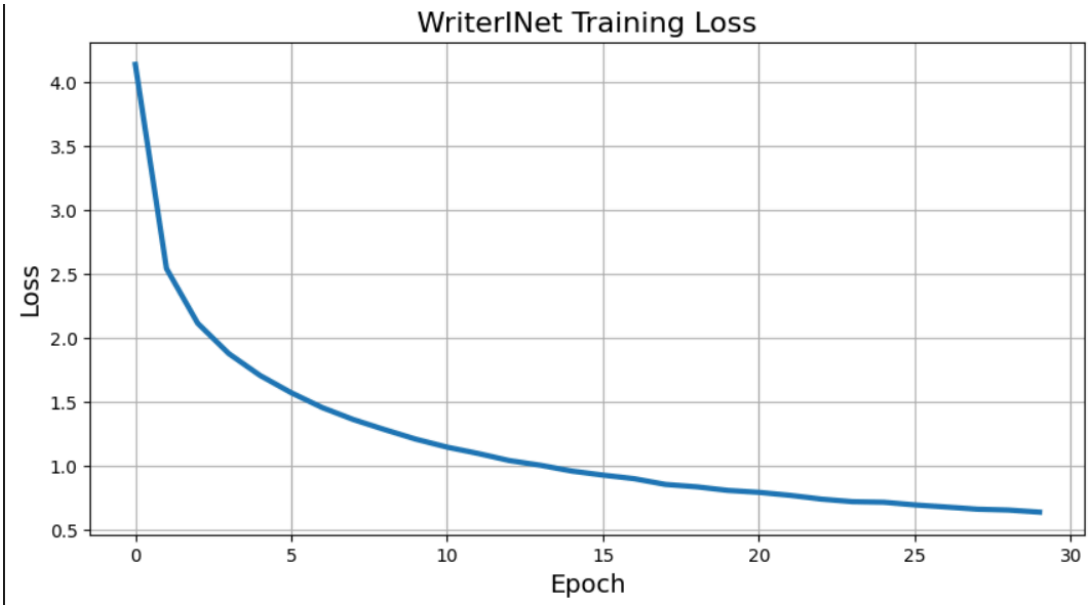
This indicates that the ANN classifier successfully learned writer-specific feature representations from the fused CNN embeddings.



14. Training Loss Curve

The training loss decreased steadily across epochs.

This shows stable optimization and convergence of the model during training.



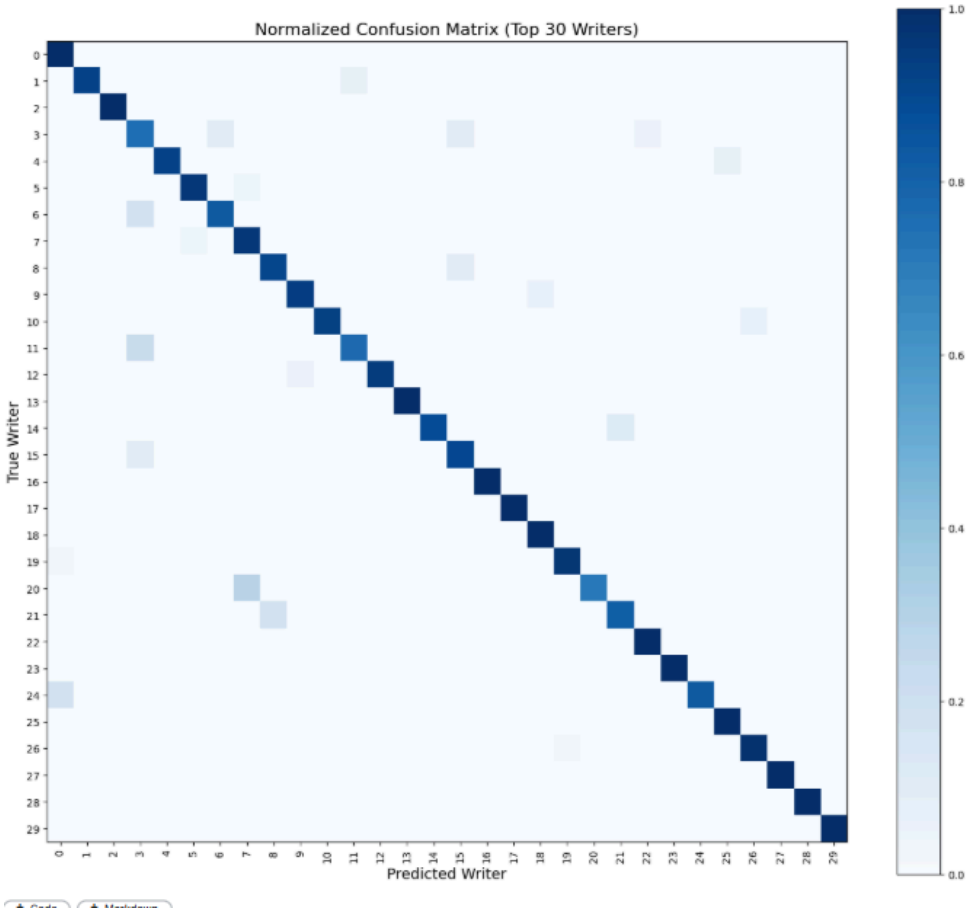
15. Confusion Matrix Analysis

A normalized confusion matrix was generated for the top 30 writer classes.

Observations:

- Strong diagonal dominance was observed
- Most writers were classified correctly
- Small confusion existed between visually similar handwriting styles

The confusion matrix confirms that the feature fusion model learned discriminative handwriting representations.



16. Conclusion

In this project, WriterINet was successfully implemented using DenseNet201 and ResNet50 feature fusion for writer identification.

The IAM Handwriting Dataset containing 657 writers was used for training and evaluation. Feature extraction was performed using pretrained CNN models, and the fused features were classified using an ANN classifier.

To improve computational efficiency, feature caching optimization was implemented, significantly reducing training time.

The final system achieved a test accuracy of 61.66%, demonstrating the effectiveness of deep feature fusion for writer identification tasks.

17. References

1. IAM Handwriting Dataset
2. PyTorch Documentation
3. DenseNet201 Architecture

4. ResNet50 Architecture
5. Scikit-learn Documentation